

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

КУРСОВАЯ РАБОТА
по дисциплине «Алгоритмы и структуры данных»
Тема: Выбор оптимальной структуры данных

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Иванов Д.В.

Санкт-Петербург

2025

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Студентка Киселева А.Д.

Группа 4342

Тема работы: Выбор оптимальной структуры данных

Исходные данные:

Вариант 4.1

В столовой “Мария” решили ввести функцию онлайн заказа еды. Для этого необходимо хранить название блюда и его цену. Для удобства выбора нужно уметь выводить список позиций по названию (в лексикографическом порядке). Также студенты желают иметь возможность поиска цены блюда по его названию. При этом обновление ассортимента позиций происходит регулярно, но реже запросов на доступ к списку блюд. Выберите и реализуйте оптимальную структуру данных для введения новой функции. Для выбора структуры данных нужно провести подробное исследование и зафиксировать его в отчете. Для демонстрации работы необходимо сделать промежуточные выводы работы программы и визуализацию структуры данных.

Содержание пояснительной записки:

«Аннотация», «Содержание», «Введение», «Выбор структуры данных», «Реализация структур и алгоритмов», «Исследование», «Заключение», «Список использованных источников»

Предполагаемый объем пояснительной записки:

Не менее 10 страниц.

Дата выдачи задания: 24.11.2025

Дата сдачи реферата: 20.12.2025

Дата защиты реферата: 22.12.2025

Студентка

Киселева А.Д.

Преподаватель

Иванов Д.В.

АННОТАЦИЯ

Курсовая работа посвящена разработке консольной утилиты для системы онлайн-заказа еды в столовой "Мария". Программа реализует два режима работы: администраторский и клиентский, предоставляя функционал для управления меню блюд и формирования заказов. В системе администратор может добавлять, удалять и изменять цены блюд в меню, в то время как клиенты имеют возможность просматривать меню, выбирать блюда для заказа, управлять содержимым корзины и рассчитывать итоговую стоимость. Чтобы программа одинаково быстро работала при различных количествах позиций меню, производится исследование подходящих структур данных и в соответствие с временем выполнения каждой операции выбирается оптимальная структура.

SUMMARY

The course work is devoted to the development of a console utility for the online food ordering system in the Maria canteen. The program implements two modes of operation: admin and client, providing functionality for managing the menu of dishes and forming orders. In the system, the administrator can add, delete and change the prices of dishes in the menu, while customers have the opportunity to view the menu, select dishes to order, manage the contents of the basket and calculate the total cost. In order for the program to work equally quickly with different numbers of menu items, suitable data structures are investigated and the optimal structure is selected in accordance with the execution time of each operation.

СОДЕРЖАНИЕ

	Введение	6
1.	Выбор структуры данных	7
2.	Реализация структур и алгоритмов	13
3.	Исследование	19
	Заключение	22
	Список использованных источников	23
	Приложение А. Тестирование	24
	Приложение Б. Код программы	26
	Приложение С. Примеры работы программы	34

ВВЕДЕНИЕ

Цель: разработать программу для системы онлайн-заказа еды в столовой «Мария», обеспечивающую эффективное обновление, быстрый поиск и сортированный вывод данных о блюдах с учётом специфики частоты операций обновления и чтения.

Задачи:

1. Провести сравнительный анализ структур данных для выбора оптимального решения, соответствующего требованиям задачи.
2. Реализовать базовые операции с выбранной структурой данных: добавление, удаление, изменение цены блюда, поиск по названию и лексикографический вывод всего меню.
3. Организовать взаимодействие с пользователем через консольный интерфейс с поддержкой двух режимов работы (администратор и клиент) и набором текстовых команд.
4. Исследовать временную сложность каждой операции для всех случаев и отразить ее на графиках зависимости времени от размера входных данных.
5. Протестировать программу.

1. ВЫБОР СТРУКТУРЫ ДАННЫХ

1.1. Формализация постановки задачи

1.1.1. Операции

В рамках решаемой задачи структура должна поддерживать следующие операции:

1. Добавление элемента по ключу и значению;
2. Обновление значения существующего ключа;
3. Поиск значения элемента по ключу;
4. Удаление элемента по ключу;
5. Вывод всех элементов в лексикографическом порядке.

1.1.2 Анализ частоты операций

Согласно заданию, обновление списка элементов происходит регулярно, но реже запросов на вывод всех элементов в отсортированном порядке и поиска элемента по ключу.

1.1.3 Характеристики данных

Ожидаемый объем входных данных не превышает 1000 позиций, предполагается активное изменение данных. Ключи имеют следующие характеристики:

- Тип: строка, содержащая название блюда;
- Характер доступа: случайный (поиск конкретного блюда) + последовательный (вывод всего меню);
- Уникальность: все ключи уникальны, не может быть двух блюд с одинаковым названием.

1.1.4 Критерии выбора

1. Эффективность поиска по ключу, так как это наиболее частая операция.
2. Поддержка вывода позиций в лексикографическом порядке, вторая по частоте операция.
3. Эффективность вставки и удаления — операции выполняются регулярно.

1.2 Анализ альтернативных структур данных

1.2.1 Массив

Массив — структура данных, которая хранит набор однородных данных, доступ к которым осуществляется по индексам. Элементы массива хранятся последовательно, что требует выделения непрерывного участка памяти.

Для наиболее эффективной операции поиска $O(\log n)$ массив должен быть отсортирован по названиям блюд. Для операций вставки и удаления в среднем случае нужно сдвинуть $n/2$ элементов, в худшем n элементов массива, что оценивается как $O(n)$. При большом количестве блюд будут потери в скорости.

Для вывода меню в лексикографическом порядке требуется обойти весь отсортированный массив, что займет $O(n)$ времени.

Исходя из рассуждений выше, массив не подходит для данной задачи из-за отсутствия возможности одновременно быстрого поиска и обновления позиций меню.

1.2.2 Хэш-таблица

Хэш-таблица — структура данных, которая хранит пары ключ-значение, используя хэш-функцию для вычисления индекса в массиве. Коллизии разрешаются методом цепочек или открытой адресации.

Для операции поиска по названию блюда в среднем случае требуется $O(1)$ время, так как хэш-функция равномерно распределяет ключи по ячейкам. Однако в худшем случае, при плохой хэш-функции или искусственно

подобранных данных, все ключи могут попасть в одну ячейку, что приведёт к поиску по списку из n элементов за $O(n)$.

Операции вставки и удаления также имеют среднюю сложность $O(1)$, но в худшем случае требуют $O(n)$ из-за необходимости обработки длинной цепочки коллизий или рехэширования всей таблицы при высокой нагрузке.

Для вывода меню в лексикографическом порядке требуется дополнительная сортировка всех элементов, что занимает $O(n \log n)$ времени, поскольку хэш-таблица не сохраняет порядок ключей.

Хэш-таблица не подходит для данной задачи, так как не поддерживает упорядоченный вывод данных без дополнительной затратной сортировки, что противоречит требованию частого отображения меню в отсортированном виде.

1.2.3 Бинарное дерево

Бинарное дерево поиска (BST) — структура данных, в которой каждый узел содержит ключ и два поддерева (левое и правое), причём все ключи в левом поддереве меньше, а в правом — больше ключа родителя.

Для операции поиска по названию блюда в среднем случае требуется $O(\log n)$ время при сбалансированном дереве. Однако в худшем случае, когда ключи добавляются в отсортированном порядке, дерево вырождается в список, и поиск занимает $O(n)$.

Операции вставки и удаления также имеют среднюю сложность $O(\log n)$, но в худшем случае требуют $O(n)$ из-за необходимости обхода вырожденного дерева.

Вывод меню в лексикографическом порядке осуществляется за $O(n)$ с помощью централизованного обхода дерева.

BST не подходит для данной задачи, так как не гарантирует сбалансированность, что может привести к деградации производительности до $O(n)$ при определённых последовательностях вставки.

1.2.4 AVL-дерево

AVL-дерево — сбалансированное бинарное дерево поиска, в котором для каждого узла высота его поддеревьев различается не более чем на 1, что обеспечивается операциями поворота после вставки и удаления.

Для операции поиска по названию блюда гарантируется время $O(\log n)$ как в среднем, так и в худшем случае благодаря строгой балансировке дерева.

Операции вставки и удаления также выполняются за $O(\log n)$, поскольку после модификации может потребоваться балансировка от корня до изменённого узла.

Вывод меню в лексикографическом порядке осуществляется за $O(n)$ с помощью центрированного обхода дерева.

AVL-дерево подходит для данной задачи, так как обеспечивает гарантированно логарифмическое время для всех основных операций, поддерживает данные в отсортированном порядке и эффективно справляется с профилем нагрузки, где поиск и вывод происходят значительно чаще обновлений.

1.2.5 Красно-черное дерево

Красно-чёрное дерево — сбалансированное бинарное дерево поиска, в котором каждый узел имеет цвет (красный или чёрный), и выполняются свойства, ограничивающие высоту дерева, обеспечивая приближённую балансировку.

Для операции поиска по названию блюда гарантируется время $O(\log n)$ как в среднем, так и в худшем случае, поскольку дерево остаётся сбалансированным за счёт соблюдения свойств красно-чёрного дерева.

Операции вставки и удаления также выполняются за $O(\log n)$, так как требуют коррекции цвета и выполнения поворотов, количество которых ограничено константой, что сохраняет логарифмическую сложность.

Вывод меню в лексикографическом порядке осуществляется за $O(n)$ с помощью централизованного обхода дерева, что соответствует требованию задачи.

Красно-чёрное дерево подходит для данной задачи, однако АВЛ-дерево является более предпочтительным выбором, поскольку обеспечивает более строгую балансировку, что даёт меньшую константу в операциях поиска при частоте запросов на чтение, значительно превышающей частоту обновлений.

1.2.6. Выбор структуры

Для наглядности и удобства выбора сложности всех операций структур, рассмотренных выше, отображены в Таблице 1.

Таблица 1 — Сравнение сложности операций на различных структурах данных

Структура данных	Поиск (средний/ худший случаи)	Вставка/ Удаление (средний/ худший случаи)	Упорядочен ный вывод	Комментарий
Отсортированный массив	$O(\log n)$	$O(n)$	$O(n)$	Требуется поддержка в отсортированном виде для обеспечения бинарного поиска, вставка/удаление потребуют сдвига элементов.
Хэш-таблица	$O(1)/O(n)$	$O(1)/O(n)$	$O(n \log n)$	Для вывода меню в лексикографическом порядке потребуется сортировка.
Бинарное дерево	$O(\log n)/O(n)$	$O(\log n)/O(n)$	$O(n)$	Высокий риск низкой скорости из-за несбалансированности дерева по высоте.

АВЛ-дерево	$O(\log n)$	$O(\log n)$	$O(n)$	При частых запросах на обновление элементов тратится время на перебалансировку.
Красно-чёрное дерево	$O(\log n)$	$O(\log n)$	$O(n)$	При частых запросах на вывод всех элементов тратится больше времени из-за нестрогой сбалансированности по высоте.

В соответствие с проведенным анализом, в качестве основной структуры данных выбрано АВЛ-дерево, так как оно полностью удовлетворяет условиям задачи: обеспечивает быстрый вывод позиций меню в лексикографическом порядке, поиск позиции и вставку/удаление элементов. Время, затрачиваемое на перебалансировку не является критичным, так как операции вывода и поиска производятся чаще, чем операции обновления ассортимента.

2. РЕАЛИЗАЦИЯ СТРУКТУР И АЛГОРИТМОВ

Структура *Node* предназначена для хранения содержимого узла АВЛ-дерева.

Поля:

- *std::string name* — название блюда;
- *int cost* — цена блюда;
- *int height* — высота узла;
- *Node* left* — левый ребенок;
- *Node* right* — правый ребенок.

Методы:

1. *Node(std::string name, int cost)* — конструктор с параметрами, используется для создания узла с определенным названием и ценой.

Класс *AvlTree* представляет реализацию всего функционала АВЛ-дерева.

Поля:

- *Node* rootNode* — указатель на корень дерева;
- *int resultCost* — поле, которое используется для подсчета итоговой стоимости товаров в корзине.

Методы:

1. *Node* balance(Node* root)* — метод балансировки дерева. Если абсолютное значение баланс-фактора узла больше единицы, то выполняются соответствующие повороты, чтобы обеспечить сбалансированную высоту дерева.

Принимаемый аргумент:

Node root* — указатель на узел, относительно которого должна происходить балансировка.

Возвращаемое значение:

Node node* — указатель на узел сбалансированного поддерева.

2. *int getBF(Node* root)* - метод вычисления баланс-фактора узла. Вычисляет разницу высот правого и левого поддеревьев для определения необходимости балансировки.

Принимаемый аргумент:

Node root* — указатель на узел, для которого вычисляется баланс-фактор.

Возвращаемое значение:

int balanceFactor — целочисленное значение баланс-фактора.

3. *void update(Node* root)* — метод обновления высоты узла. Пересчитывает высоту узла на основе высот его дочерних поддеревьев.

Принимаемый аргумент:

Node root* — указатель на узел, высоту которого необходимо обновить.

4. *Node* rotateLeft(Node* root)* — метод левого поворота вокруг указанного узла. Используется для балансировки дерева при перевесе правого поддерева.

Принимаемый аргумент:

Node root* — указатель на узел, вокруг которого выполняется поворот.

Возвращаемое значение:

Node node* — указатель на новый корень поддерева после поворота.

5. *Node* rotateRight(Node* root)* — метод правого поворота вокруг указанного узла. Используется для балансировки дерева при перевесе левого поддерева.

Принимаемый аргумент:

Node root* — указатель на узел, вокруг которого выполняется поворот.

Возвращаемое значение:

Node node* — указатель на новый корень поддерева после поворота.

6. *Node* minimum(Node* root)* — метод поиска узла с минимальным ключом в поддереве. Находит блюдо с наименьшим названием в лексикографическом порядке.

Принимаемый аргумент:

Node root* — указатель на корень поддерева для поиска.

Возвращаемое значение:

Node node* — указатель на узел с минимальным ключом.

7. *Node* deleteNode(Node* root, std::string key)* — метод рекурсивного удаления узла из поддерева. Удаляет узел с указанным ключом, затем балансирует поддерево.

Принимаемые аргументы:

Node root* — указатель на корень поддерева для удаления.

std::string key — название блюда для удаления.

Возвращаемое значение:

Node node* — указатель на корень сбалансированного поддерева после удаления.

8. *Node* insertNode(Node* root, std::string name, int cost)* — метод рекурсивной вставки узла в поддерево. Добавляет новый узел или обновляет стоимость существующего блюда, затем балансирует поддерево.

Принимаемые аргументы:

Node root* — указатель на корень поддерева для вставки.

std::string name — название блюда (ключ).

int cost — цена блюда.

Возвращаемое значение:

Node node* — указатель на корень сбалансированного поддерева после вставки.

9. *Node* search(Node* root, std::string name)* — метод рекурсивного поиска узла по ключу. Находит узел с указанным названием блюда в поддерева.

Принимаемые аргументы:

Node root* — указатель на корень поддерева для поиска.

std::string name — название блюда для поиска.

Возвращаемое значение:

Node node* — указатель на найденный узел или *nullptr*, если узел не найден.

10. *void inorderTraversal(Node* node)* — метод центрированного обхода поддерева. Выводит все блюда в лексикографическом порядке и вычисляет суммарную стоимость для корзины.

Принимаемый аргумент:

Node node* — указатель на корень поддерева для обхода.

11. *void destroyTree(Node* node)* — метод рекурсивного удаления всех узлов дерева. Освобождает память, выделенную для узлов поддерева.

Принимаемый аргумент:

Node node* — указатель на корень поддерева для удаления.

12. *AvlTree()* — конструктор по умолчанию, инициализирует указатель на корень дерева как *nullptr*, итоговую сумму блюд в корзине как 0.

13. *void insert(std::string name, int cost)* — публичный метод вставки блюда в дерево. Обеспечивает добавление нового блюда или обновление цены существующего.

Принимаемые аргументы:

std::string name — название блюда.

int cost — цена блюда.

14. *void remove(std::string name)* — публичный метод удаления блюда из дерева. Удаляет блюдо с указанным названием, если оно существует.

Принимаемый аргумент:

std::string name — название удаляемого блюда.

15. *void show(std::string item)* — публичный метод вывода содержимого дерева. Выводит все блюда в лексикографическом порядке с ценами, при выводе корзины также показывает итоговую стоимость.

Принимаемый аргумент:

std::string item — контекст вывода ("*menu*" для меню, "*cart*" для корзины).

16. *int searchByName(std::string name)* — публичный метод поиска цены блюда по названию. Возвращает цену блюда или -1, если блюдо не найдено.

Принимаемый аргумент:

std::string name — название искомого блюда.

Возвращаемое значение:

int cost — цена найденного блюда или -1 при ошибке.

17. *~AvlTree()* - деструктор, удаляет все дерево и освобождает выделенную память.

Класс *Menu* реализует взаимодействие пользователя с программой: обрабатывает ввод, позволяет производить операции с содержанием меню и корзины, определяет права доступа к определенным функциям в зависимости от типа текущего пользователя.

Поля:

- *User user* — поле, определяющее тип пользователя;
- *AvlTree menu* — объект меню;

- *AvlTree customer* — объект корзины;
- *int KEY* — пароль для авторизации администратора и получения доступа к командам модификации меню.

Методы:

1. *void help()* — метод вывода справки, содержащей доступные команды для определенного типа пользователя.
2. *void checkRights()* - метод проверки прав пользователя. Изначально пользователь определен как клиент, чтобы поменять тип на администратора, нужно ввести пароль. Команды клиента и администратора различаются: клиент может только управлять корзиной, администратор — только редактировать меню.
3. *int stringToInt(std::string str)* — метод преобразования строки в целое число.

Принимаемый аргумент:

std::string str — строка, которую нужно преобразовать.

Возвращаемое значение:

int value — число, полученное из строки.

4. *Menu()* — конструктор по умолчанию, устанавливает пользователя как клиента, создает объекты меню и корзины.
5. *void start()* — метод запуска программы. Обработывает ввод пользователя, вызывает соответствующие методы или выводит сообщения об ошибках.

В функции *main()* создается объект класса *Menu* и запускается программа с помощью вызова у объекта соответствующего метода.

3. ИССЛЕДОВАНИЕ

Для оценки временной сложности операций AVL-дерева произведено исследование времени выполнения функций вставки и удаления элементов, поиска по названию и централизованного обхода дерева. Теоретическая сложность операций:

1. Вставка элемента во всех случаях оценивается как $O(\log n)$, так как в лучшем случае при вставке элемента в лист нужно пройти до него $\log n$ узлов, в среднем и худшем случае понадобятся перебалансировки на пути к корню.
2. Удаление элемента также имеет сложность $O(\log n)$ во всех случаях, поскольку требует поиска удаляемого узла за $O(\log n)$, а в случае узла с двумя потомками — дополнительного поиска минимального элемента в правом поддереве за $O(\log n)$ с последующей балансировкой на пути к корню.
3. Поиск элемента в лучшем случае выполняется за $O(1)$ при условии, что он находится в корне дерева. В среднем и худшем случаях благодаря сбалансированности дерева операция оценивается в $O(\log n)$.
4. Централизованный обход дерева всегда занимает $O(n)$ времени, так как необходимо посетить каждый узел ровно один раз.

Выполнялся замер времени для деревьев, содержащих от 10 до 20000 случайных элементов. Результаты исследования отражены на графиках зависимости объема входных данных от времени выполнения операций (см. Рисунок 1 и Рисунок 2). Ожидаемые оценки временной сложности совпали с полученными на практике: операции вставки, удаления и поиска по графикам находятся ниже пунктирной теоретической прямой, операция централизованного

обхода колеблется в окрестности теоретической прямой.

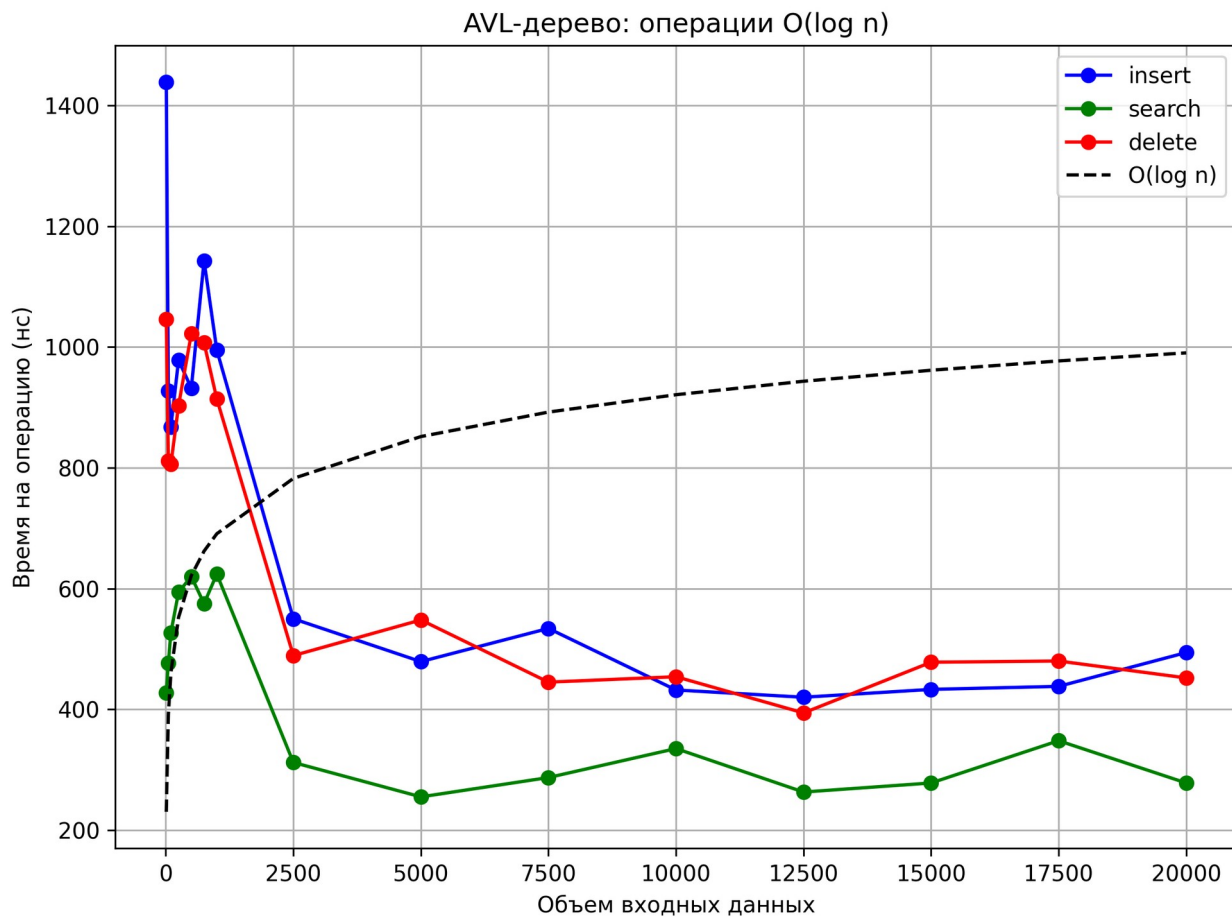


Рисунок 1 — График операций вставки, удаления и поиска элементов

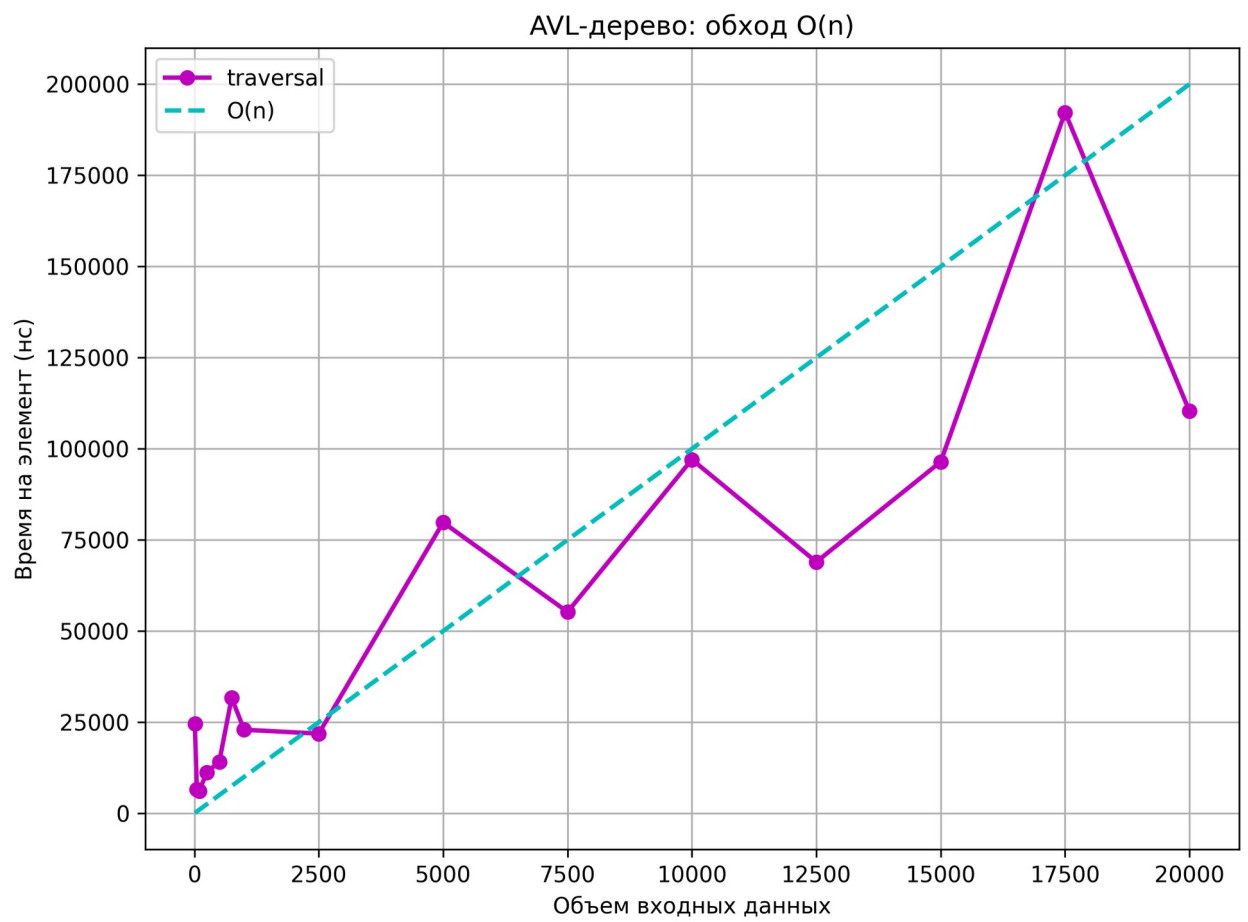


Рисунок 2 — График операции центрированного обхода

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана консольная программа для системы онлайн-заказа еды в столовой «Мария» на основе АВЛ-дерева. Программа успешно реализует все поставленные задачи: управление меню блюд (добавление, удаление, изменение цены), поиск блюд по названию и вывод всего меню в лексикографическом порядке.

Проведённый анализ альтернативных структур данных показал, что АВЛ-дерево является оптимальным выбором для данной задачи. Оно обеспечивает гарантированную логарифмическую сложность $O(\log n)$ для операций вставки, удаления и поиска, что соответствует профилю нагрузки системы, где операции чтения (поиск и вывод меню) выполняются значительно чаще операций обновления. Экспериментальные исследования подтвердили теоретические оценки сложности операций.

Разработанная программа демонстрирует стабильную работу, корректную обработку пользовательского ввода, разграничение прав доступа между администратором и клиентом, а также эффективное использование памяти. Интерфейс программы интуитивно понятен и включает контекстную справку по командам.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ахо А., Хопкрофт Дж., Ульман Дж. Структуры данных и алгоритмы. — М.: Вильямс, 2001. — 384 с.
2. Седжвик Р. Фундаментальные алгоритмы на C++. — М.: Диасофт, 2002. — 688 с.
3. Липпман С., Лажойе Ж., Му Б. Язык программирования C++. — М.: Вильямс, 2014. — 1184 с.

ПРИЛОЖЕНИЕ А

ТЕСТИРОВАНИЕ

№ теста	Тест	Входные данные	Выходные данные	Комментарий
1	Вставка одного элемента	insert("Pizza", 350)	Найден элемент Pizza = 350	Базовая проверка вставки
2	Поиск несуществующего элемента	searchByName("Burger") в пустом дереве	-1	Корректная обработка отсутствующего элемента
3	LL балансировка	insert("C", 300) → insert("B", 200) → insert("A", 100)	Все элементы найдены: A=100, B=200, C=300	Проверка правого вращения при левом перевесе
4	RR балансировка	insert("A", 100) → insert("B", 200) → insert("C", 300)	Все элементы найдены: A=100, B=200, C=300	Проверка левого вращения при правом перевесе
5	LR балансировка	insert("C", 300) → insert("A", 100) → insert("B", 200)	Все элементы найдены: A=100, B=200, C=300	Проверка двойного вращения (влево, затем вправо)
6	RL балансировка	insert("A", 100) → insert("C", 300) → insert("B", 200)	Все элементы найдены: A=100, B=200, C=300	Проверка двойного вращения (вправо, затем влево)
7	Удаление узла с двумя потомками	insert("B", 200) → insert("A", 100) → insert("C", 300) → remove("B")	A=100, C=300 найдены; B=-1	Проверка удаления с заменой на минимум правого поддерева
8	Удаление несуществующего элемента	insert("Pizza", 350) → remove("Burger")	Удаляемый элемент не найден	Корректная обработка удаления отсутствующего элемента
9	Обновление цены существующего блюда	insert("Pizza", 350) → insert("Pizza", 400)	Pizza = 400	Повторный insert обновляет значение, узел не создаётся
10	Множественная вставка и поиск	Вставка 5 блюд: Pizza, Burger, Salad, Pasta, Soup	Все 5 блюд найдены с правильными ценами	Проверка работы с множественными элементами
11	Последовательное добавление и удаление	insert("Item0"-"Item4") → поиск → remove("Item0"-"Item4")	Все элементы найдены при добавлении, все удалены после	Проверка полного цикла добавления-удаления

			remove	
12	Пустая строка как ключ	insert("",100) → searchByName("") → remove("")	Вставка не произведена	Проверка граничного случая с пустой строкой

```

[=====] Running 12 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 12 tests from AVLTreeTest
[ RUN    ] AVLTreeTest.InsertSingleElement
[ OK     ] AVLTreeTest.InsertSingleElement (0 ms)
[ RUN    ] AVLTreeTest.SearchNonExistent
Search error: dish is not on the menu.
[ OK     ] AVLTreeTest.SearchNonExistent (0 ms)
[ RUN    ] AVLTreeTest.LeftLeftRotation
[ OK     ] AVLTreeTest.LeftLeftRotation (0 ms)
[ RUN    ] AVLTreeTest.RightRightRotation
[ OK     ] AVLTreeTest.RightRightRotation (0 ms)
[ RUN    ] AVLTreeTest.LeftRightRotation
[ OK     ] AVLTreeTest.LeftRightRotation (0 ms)
[ RUN    ] AVLTreeTest.RightLeftRotation
[ OK     ] AVLTreeTest.RightLeftRotation (0 ms)
[ RUN    ] AVLTreeTest.DeleteNodeWithTwoChildren
Search error: dish is not on the menu.
[ OK     ] AVLTreeTest.DeleteNodeWithTwoChildren (0 ms)
[ RUN    ] AVLTreeTest.DeleteNonExistent
Delete error: dish not exist.
[ OK     ] AVLTreeTest.DeleteNonExistent (0 ms)
[ RUN    ] AVLTreeTest.UpdateExistingItem
[ OK     ] AVLTreeTest.UpdateExistingItem (0 ms)
[ RUN    ] AVLTreeTest.MultipleInsertAndSearch
[ OK     ] AVLTreeTest.MultipleInsertAndSearch (0 ms)
[ RUN    ] AVLTreeTest.SequentialAddRemove
Search error: dish is not on the menu.
Search error: dish is not on the menu.
Search error: dish is not on the menu.
Search error: dish is not on the menu.
Search error: dish is not on the menu.
[ OK     ] AVLTreeTest.SequentialAddRemove (0 ms)
[ RUN    ] AVLTreeTest.EmptyStringKey
Search error: dish is not on the menu.
[ OK     ] AVLTreeTest.EmptyStringKey (0 ms)
[-----] 12 tests from AVLTreeTest (0 ms total)

[-----] Global test environment tear-down
[=====] 12 tests from 1 test suite ran. (0 ms total)
[ PASSED ] 12 tests.

```

ПРИЛОЖЕНИЕ Б

КОД ПРОГРАММЫ

Файл menu.hpp

```
#ifndef MENU_H
#define MENU_H

#include "avltree.hpp"

enum User{
    ADMIN,
    CUSTOMER
};

class Menu{
    User user;
    AvlTree menu;
    AvlTree customer;
    int KEY = 1503;

    void help();
    void checkRights();
    int stringToInt(std::string str);

public:
    Menu();
    void start();
};

#endif
```

Файл menu.cpp

```
#include "menu.hpp"

Menu::Menu() : user(User::CUSTOMER), menu(AvlTree()), customer(AvlTree())
{}

void Menu::help(){
    switch (user){
        case ADMIN:
            std::cout << "\nADMIN COMMAND SET\n";
            std::cout << "del <name> -- delete dish from the menu.\n";
            std::cout << "add <name> <price> -- add dish to the menu.\n";
            std::cout << "add <existing_name> <new_price> -- if you want
to change th price of the existing dish.\n";
            break;

        case CUSTOMER:
```

```

        std::cout << "\nCUSTOMER COMMAND SET\n";
        std::cout << "select <name> -- add dish to cart.\n";
        std::cout << "del <name> -- delete dish from the cart.\n";
        std::cout << "cart -- show cart contents and total cost.\n";
        break;

        default: break;
    }
    std::cout << "menu -- show all available menu dishes.\n";
    std::cout << "search <name> -- search price by dishname in the menu.\n";
    std::cout << "help -- show command set.\n";
    std::cout << "exit -- close program.\n";
    std::cout << "admin -- if you are an admin, enter this command.\n\n";
}

int Menu::stringToInt(std::string str){
    int res = 0;
    for (char symb : str){
        if (symb > '9' || symb < '0')
            return -1;
        res = res*10 + (symb - '0');
    }
    return res;
}

void Menu::checkRights(){
    if (user != ADMIN){
        std::string code;
        std::cout << "Password: ";
        if (std::getline(std::cin, code)){
            if (stringToInt(code) == KEY){
                std::cout << "Success!\nIf you want to return to the
customer mode, enter 'admin' again.\n";
                user = ADMIN;
            } else std::cout << "Wrong password, try again.\n";
        }
    } else {
        std::cout << "Success! Change user to customer.\n";
        user = CUSTOMER;
    }
}

void Menu::start(){
    std::string inputData;
    help();

    while (std::getline(std::cin, inputData)){
        if (inputData.empty()) continue;

        int firstSpace = (int)inputData.find(' ');

        if (firstSpace == (int)std::string::npos){

```

```

        if (inputData == "menu") menu.show(inputData);
        else if (inputData == "exit") break;
        else if (inputData == "help") help();
        else if (inputData == "admin") checkRights();
        else if (inputData == "cart" && user == CUSTOMER)
customer.show(inputData);
        else std::cout << "Unknown command. Print 'help' to get more
information.\n";

    } else {
        std::string command = inputData.substr(0, firstSpace);
        std::string args = inputData.substr(firstSpace + 1);

        if (command == "search"){
            if (!args.empty()){
                int cost = menu.searchByName(args);
                if (cost != -1) std::cout << "> " << args << " costs
" << cost << ".\n";
            } else std::cout << "Search error: enter the dishname.\n";
        }

        } else if (command == "del"){
            if (!args.empty()){
                if (user == ADMIN) menu.remove(args);
                else if (user == CUSTOMER) customer.remove(args);
                else std::cout << "User not defined.\n";
            }
            else std::cout << "Delete error: enter the dishname.\n";
        }

        } else if (command == "add" && user == ADMIN){
            int lastSpace = args.find_last_of(' ');

            if (lastSpace == (int)std::string::npos)
                std::cout << "Add error: enter the dishname and
price.\n";
            else {
                std::string dishName = args.substr(0, lastSpace);
                int cost = stringToInt(args.substr(lastSpace + 1));

                if (dishName.empty())
                    std::cout << "Add error: dishname can't be
skipped.\n";
                else if (cost == -1)
                    std::cout << "Add error: invalid cost.\n";
                else menu.insert(dishName, cost);
            }
        }

        } else if (command == "select" && user == CUSTOMER){
            if (!args.empty()){
                int cost = menu.searchByName(args);
                if (cost != -1) customer.insert(args, cost);
            }
        }
    }
}

```

```

        } else std::cout << "Select error: enter the dishname.\n";
    }
    } else std::cout << "Unknown command. Print 'help' to get
more information.\n";
    }
}
}

```

Файл avltree.hpp

```

#ifndef MENU_TREE_H
#define MENU_TREE_H

#include <string>
#include <iostream>

struct Node{
    std::string name;
    int cost;
    int height;
    Node* left;
    Node* right;

    Node(std::string name, int cost) :
        name(name), cost(cost), height(1), left(nullptr), right(nullptr)
    {}
};

class AvlTree{
    Node* rootNode;
    int resultCost;

    Node* balance(Node* root);
    int getBF(Node* root);
    void update(Node* root);
    Node* rotateLeft(Node* root);
    Node* rotateRight(Node* root);
    Node* minimum(Node* root);
    Node* deleteNode(Node* root, std::string key);
    Node* insertNode(Node* root, std::string name, int cost);
    Node* search(Node* root, std::string name);
    void inOrderTraversal(Node* node);
    void destroyTree(Node* node);

public:
    AvlTree();
    void insert(std::string name, int cost);
    void remove(std::string name);
    void show(std::string item);
    int searchByName(std::string name);
    ~AvlTree();
};

```

```
#endif
```

Файл avltree.cpp

```
#include "avltree.hpp"
```

```
AvlTree::AvlTree():rootNode(nullptr), resultCost(0) {}
```

```
Node* AvlTree::balance(Node* root){
    if (!root)
        return root;
    update(root);
    int balanceFactor = getBF(root);

    if (balanceFactor < -1){
        if (getBF(root->left) > 0)
            root->left = rotateLeft(root->left);
        return rotateRight(root);
    }
    else if (balanceFactor > 1){
        if (getBF(root->right) < 0)
            root->right = rotateRight(root->right);
        return rotateLeft(root);
    }

    return root;
}

int AvlTree::getBF(Node* root){
    if (!root) return 0;
    int leftHeight = root->left ? root->left->height : 0;
    int rightHeight = root->right ? root->right->height : 0;
    return rightHeight - leftHeight;
}

void AvlTree::update(Node* root){
    if (!root) return;
    int lh = (root->left) ? root->left->height : 0;
    int rh = (root->right) ? root->right->height : 0;
    root->height = (lh > rh) ? lh + 1 : rh + 1;
}

Node* AvlTree::rotateLeft(Node* root){
    Node* rightChild = root->right;

    root->right = rightChild->left;
    rightChild->left = root;

    update(root);
    update(rightChild);
    return rightChild;
}
```

```

Node* AvlTree::rotateRight(Node* root){
    Node* leftChild = root->left;

    root->left = leftChild->right;
    leftChild->right = root;

    update(root);
    update(leftChild);
    return leftChild;
}

Node* AvlTree::insertNode(Node* root, std::string name, int cost){
    if (root == nullptr){
        root = new Node(name, cost);
        return root;
    }
    if (name.compare(root->name) < 0)
        root->left = insertNode(root->left, name, cost);
    else if (name.compare(root->name) > 0)
        root->right = insertNode(root->right, name, cost);
    else root->cost = cost;

    return balance(root);
}

Node* AvlTree::deleteNode(Node* root, std::string key){
    if (!root) return root;
    if (key.compare(root->name) < 0)
        root->left = deleteNode(root->left, key);
    else if (key.compare(root->name) > 0)
        root->right = deleteNode(root->right, key);
    else if (root->left != nullptr && root->right != nullptr){
        Node* minRight = minimum(root->right);
        root->name = minRight->name;
        root->cost = minRight->cost;
        root->right = deleteNode(root->right, root->name);
    } else {
        Node* tmp = root;
        if (root->left == nullptr)
            root = root->right;
        else root = root->left;
        delete tmp;
    }
    return balance(root);
}

Node* AvlTree::minimum(Node* root){
    if (root) {
        while (root->left){
            root = root->left;
        }
    }
}

```

```

        return root;
    }

Node* AvlTree::search(Node* root, std::string name){
    if (!root) return nullptr;

    if (name.compare(root->name) < 0)
        return search(root->left, name);
    else if (name.compare(root->name) > 0)
        return search(root->right, name);

    return root;
}

void AvlTree::destroyTree(Node* node){
    if (!node) return;
    destroyTree(node->left);
    destroyTree(node->right);
    delete node;
}

void AvlTree::inOrderTraversal(Node* node){
    if (!node) return;
    inOrderTraversal(node->left);
    resultCost += node->cost;
    std::cout << node->name << ' ' << "--" << ' ' << node->cost << '\n';
    inOrderTraversal(node->right);
}

void AvlTree::insert(std::string name, int cost){
    rootNode = insertNode(rootNode, name, cost);
}

void AvlTree::remove(std::string name){
    if (!search(rootNode, name))
        std::cout << "Delete error: dish not exist.\n";
    else rootNode = deleteNode(rootNode, name);
}

void AvlTree::show(std::string item){
    if (item == "menu") std::cout << "\nCANTEEN MARIA MENU\n";
    else if (item == "cart") std::cout << "\nCART\n";

    inOrderTraversal(rootNode);

    if (item == "cart") std::cout << "Result cost: " << resultCost << '\n';
    std::cout << '\n';
    resultCost = 0;
}

int AvlTree::searchByName(std::string name){
    Node* node = search(rootNode, name);
}

```



```

        if (node == nullptr){
            std::cout << "Search error: dish is not on the menu.\n";
            return -1;
        } else return node->cost;
    }

    AvlTree::~AvlTree(){
        destroyTree(rootNode);
        rootNode = nullptr;
    }

```

Файл main.cpp

```

#include "menu.hpp"

int main(){
    Menu Maria;
    Maria.start();
    return 0;
}

```

ПРИЛОЖЕНИЕ С ПРИМЕРЫ РАБОТЫ ПРОГРАММЫ

```
CUSTOMER COMMAND SET
select <name> -- add dish to cart.
del <name> -- delete dish from the cart.
cart -- show cart contents and total cost.
menu -- show all available menu dishes.
search <name> -- search price by dishname in the menu.
help -- show command set.
exit -- close program.
admin -- if you are an admin, enter this command.

admin
Password: 1503
Success!
If you want to return to the customer mode, enter 'admin' again.
help

ADMIN COMMAND SET
del <name> -- delete dish from the menu.
add <name> <price> -- add dish to the menu.
add <existing_name> <new_price> -- if you want to change th price of the existing dish.
menu -- show all available menu dishes.
search <name> -- search price by dishname in the menu.
help -- show command set.
exit -- close program.
admin -- if you are an admin, enter this command.
```

Рисунок 3 — Смена пользователя

```
add salad 15
add soup 12
add okroshka 20
add tea 5
menu

CANTEEN MARIA MENU
okroshka -- 20
salad -- 15
soup -- 12
tea -- 5
```

Рисунок 4 — Добавление блюд и вывод меню

```
del salad
menu

CANTEEN MARIA MENU
okroshka -- 20
soup -- 12
tea -- 5
```

Рисунок 5 — Удаление блюда

```
search okroshka
> okroshka costs 20.
```

Рисунок 6 — Поиск цены блюда

```
select cake
select coffee
cart

CART
cake -- 17
coffee -- 16
Result cost: 33
```

Рисунок 7 — Вывод корзины и общей стоимости покупки